Asymmetric Pursuit-Evasion Game: A Curriculum-Based PPO Approach with Adversarial Co-Training

NUID: 003186275 NUID: 002553932

1 Introduction

Pursuit-evasion (P-E) games model the adversarial dynamics between agents that seek to capture targets and agents that strive to evade. These games have attracted significant research interest in areas such as UAV coordination, robotic surveillance, and autonomous systems (Xu & Dang, 2025). Traditional game-theoretic approaches are capable of deriving optimal solutions, but they tend to limit agents' ability to discover novel strategies. On the other hand, deep reinforcement learning (DRL) methods achieve strong empirical performance, but most existing formulations assume symmetric information, meaning that all agents observe the same global state (Lehis et al., 2025). In practice, however, information asymmetry is the norm. A surveillance target cannot see camera placements, and a fugitive does not have access to police dispatch patterns.

This project develops a five-stage reinforcement learning framework for an asymmetric P-E game set on a procedurally generated 15×15 city grid. In this game, a thief agent must navigate through the city to reach a safe zone while avoiding environmental hazards and evading police agents that are equipped with CCTV surveillance. The thief perceives only a local 3×3 window around its position, while the police receive the full grid map, camera sighting data, and teammate positions. All agents are trained using Proximal Policy Optimization (PPO) through a progressive curriculum that is connected by a surgical weight transplant technique. This technique preserves previously learned behaviors as the observation space expands from 10 to 244 dimensions across stages.

1.1 Related Work

Two peer-reviewed studies provide the primary foundation for this project.

Xu & Dang (2025) investigate emergent behaviors in multi-agent P-E games on a 25×25 grid using the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm under a complete-information assumption. In their setup, two pursuers and one evader train simultaneously, and the trained pursuers achieve a 99.9% success rate across 1,000 randomized trials. Through K-means clustering of action-

sequence trajectories, they identify four cooperative pursuit strategies and document a "lazy pursuit" phenomenon in which one pursuer reduces its effort when rewards are shared equally between both agents.

Lehis et al. (2025) propose a Multi-Head DDPG architecture that decomposes each pursuer's control into three force-based behavioral primitives (pursuit, cohesion, and separation), which are combined through adaptive softmax weighting. Evaluated on a 20×20 grid with 10 pursuers and 2 evaders under partial observability, their method achieves an 85.2% success rate. Notably, this outperforms DQN at 61.2%, standard PPO at 67.4%, and vanilla DDPG at 72.6% on the same benchmark.

Our work differs from both of these studies in three key ways. First, we introduce structured information asymmetry through CCTV cameras, limited thief vision, and privileged police observations, whereas both reference works use either complete or symmetric partial information. Second, we employ a five-stage curriculum with surgical weight transplantation rather than training all agents from scratch in a single environment. Third, we focus on the evader's learning problem first, building up the thief's capabilities through progressively harder challenges before introducing learning pursuers.

2 Methods

2.1 Environment Design

The game environment is a 15×15 city grid produced by a seeded stochastic generation process. The algorithm works as follows: all border cells are initialized as walls, interior cells are randomly assigned as walls with probability $p_{\text{wall}} = 0.25$, structured corridors are carved at regular intervals of every 3 rows and 3 columns to ensure navigability, and a BFS flood-fill is applied to retain only the largest connected road component. Any isolated road pockets are converted to walls, guaranteeing that every road cell is reachable from every other road cell.

Following the Von Neumann neighborhood model adopted by both Xu & Dang (2025) and Lehis et al. (2025), each agent occupies a single grid cell and may move to one of its four cardinal neighbors or remain stationary. The discrete action space is $\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}, \text{stay}\}$, and the position update for agent i at time step t is given by:

$$\mathbf{X}_i(t+1) = \mathbf{X}_i(t) + \mathbf{u}_i(t), \quad \mathbf{u}_i(t) \in \{(\pm 1, 0),\ (0, \pm 1),\ (0, 0)\} \tag{1}$$

subject to the constraint that $\mathbf{X}_i(t+1)$ must lie on a valid road cell. The environment also includes 6 lethal traps that teleport to new positions every 50 steps, 5 bouncing traffic cars that move every 3 steps and cause a 2-step stun on contact, and 6 fixed CCTV cameras each with a 3×3 detection zone. All episodes are capped at $T_{\max} = 200$ steps.

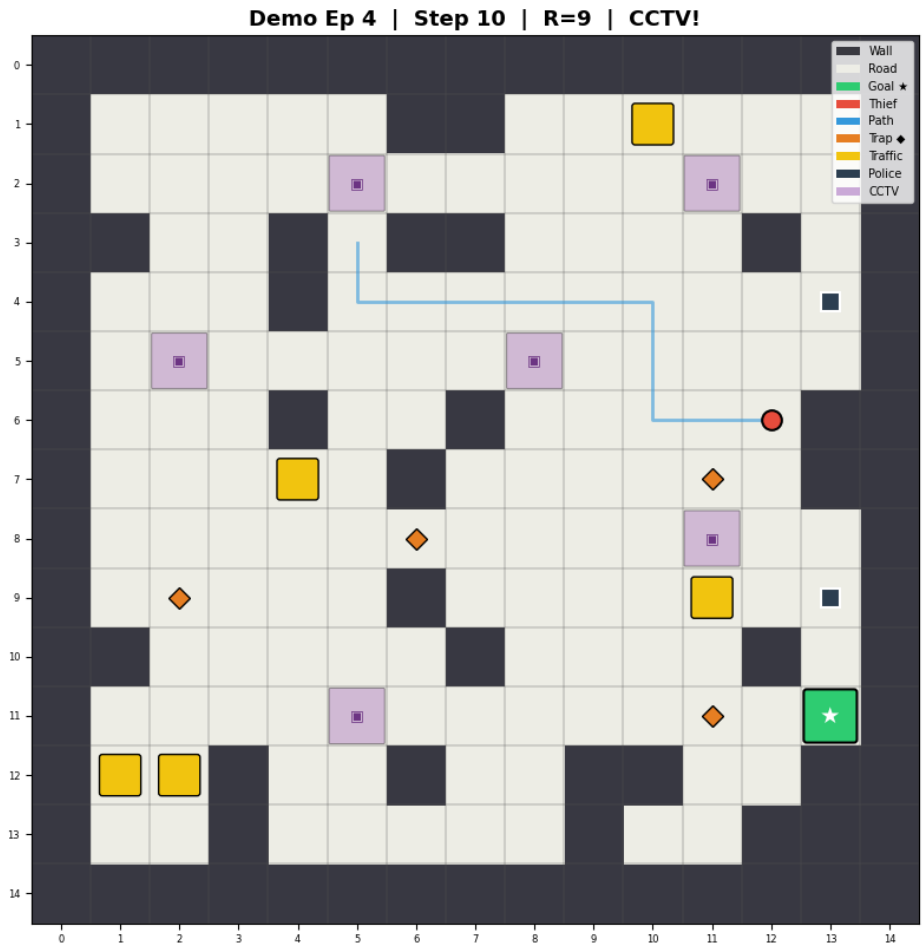

Figure 1: Example procedurally generated city grid showing walls (dark), road cells (light), CCTV cameras, traps, traffic cars, thief, police, and the safe zone.

2.2 Information Asymmetry

A central design choice in this project is the information asymmetry between the thief and police agents. Unlike the complete-information setting of Xu & Dang (2025), where all agents have full knowledge of every other agent's position at all times, our formulation gives each side a fundamentally different view of the world.

The thief perceives only a 3×3 egocentric window centered on its current position. This is analogous to the fixed sensing radius r_s used by Lehis et al. (2025), but restricted to a radius of 1 cell. The thief's observation also includes normalized Manhattan distance to the goal and, in later stages, overlays indicating the presence of traps, traffic, and police within that 3×3 window, as well as sorted distances to nearby police.

Police agents, by contrast, receive a much richer 244-dimensional observation vector. This includes

the full 15×15 grid map (225 dimensions), their own normalized position, the goal position, the last known CCTV sighting of the thief along with a staleness value indicating how many steps ago the sighting occurred, the relative position of their teammate for coordination, and a 3×3 local detection grid that indicates whether the thief is in any immediately adjacent cell.

The CCTV cameras themselves are passive sensors. When the thief passes within the 3×3 detection zone of any camera, the thief's position and current timestep are silently recorded. The thief receives no notification or reward signal when this happens, which creates a strictly one-way information channel from the thief to the police.

2.3 Algorithm Selection: PPO

The project initially used Deep Q-Networks (DQN) for training the thief agent. However, we found that DQN was insufficient for handling the high level of stochasticity introduced by dynamic traps, randomly moving traffic, and random police behavior. This difficulty is consistent with the findings of Lehis et al. (2025), who report that DQN achieves the lowest success rate (61.2%) among all algorithms they tested on grid-based pursuit-evasion tasks.

Based on this experience, we selected Proximal Policy Optimization (PPO) as the training algorithm for all agents. PPO was chosen for three reasons. First, its clipped surrogate objective prevents destructively large policy updates when reward variance is high, providing more stable learning. Second, PPO naturally supports discrete action spaces through categorical distributions, unlike DDPG which requires a continuous-to-discrete angular mapping step as described by Lehis et al. (2025). Third, PPO's on-policy nature is well suited for curriculum learning, since each stage generates fresh rollouts from the current environment without off-policy distribution shift.

The total PPO loss function used in our implementation is:

$$L(\theta) = L^{\text{CLIP}}(\theta) + c_1 \, L^{\text{VF}}(\theta) - c_2 \, H[\pi_\theta] \tag{2}$$

where L^{CLIP} is the clipped surrogate policy loss, L^{VF} is the clipped value function loss, $H[\pi_\theta]$ is the entropy of the current policy (which encourages exploration), $c_1 = 0.5$ is the value loss coefficient, and c_2 is the entropy coefficient which is linearly annealed during training.

The clipped policy loss is defined as:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\max \left(-r_t(\theta) \, \hat{A}_t, \;\; -\text{clip}\big(r_t(\theta), \, 1-\epsilon, \, 1+\epsilon\big) \, \hat{A}_t \right) \right] \tag{3}$$

where $r_t(\theta) = \pi_\theta(a_t \mid s_t) \, / \, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is the ratio between the new and old policy probabilities, $\hat{A}_t$ is the estimated advantage at step t, and $\epsilon = 0.2$ is the clipping parameter that constrains how much the policy can change in a single update.

Advantages are estimated using Generalized Advantage Estimation (GAE) with parameters $\gamma = 0.99$ and $\lambda = 0.95$. For each step t, the temporal difference residual δ_t and the GAE advantage $\hat{A}_t$ are computed

as:

$$\delta_t = r_t + \gamma\, V(s_{t+1})\,(1 - d_{t+1}) - V(s_t) \tag{4}$$

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l\, \delta_{t+l} \tag{5}$$

where r_t is the reward at step t, $V(s_t)$ is the value estimate from the critic, and d_{t+1} is a done flag that equals 1 when the episode terminates.

The value function loss also uses clipping to prevent gradient explosions through the shared network trunk. In our implementation, this clipping turned out to be critical: without it, a sudden terminal reward of +100 against a near-zero value estimate produces a squared error on the order of 10,000, which sends extremely large gradients through the shared layers and causes NaN outputs in the policy head. With value clipping, the value head can only change by at most $\pm c$ per update (where c is the value clip range), keeping gradients bounded.

2.4 Network Architecture

All agents in this project use the same actor-critic neural network structure with a shared trunk and separate output heads. The architecture consists of a 2-layer shared trunk with 128 hidden units per layer and Tanh activation functions, followed by a policy head that outputs a distribution over the 5 actions and a value head that outputs a single scalar value estimate. All layers are initialized using orthogonal initialization, which is standard practice for PPO. The policy head uses a small initialization gain of 0.01 to ensure that the initial action probabilities are approximately uniform, promoting exploration at the start of training. The network is optimized using the Adam optimizer with an epsilon value of 1×10^{-5} for numerical stability, and the learning rate is linearly annealed from its starting value to a lower final value over the course of training, as specified in Table 2.

2.5 Five-Stage Curriculum

The training pipeline is organized into five stages, each of which introduces new environmental complexity and expands the observation space. Table 1 summarizes how the observation grows across stages. Each stage loads the checkpoint from the previous stage to preserve learned behaviors.

Table 1: Observation space progression across stages.

Stage	Agent	Dim	New Components
1	Thief	10	3×3 wall patch (9) + goal distance (1)
2	Thief	28	+ trap overlay (9) + traffic overlay (9)
3	Thief	40	+ police overlay (9) + police distances (3)
4	Police	244	Full grid (225) + CCTV (4) + teammate (2) + detection (9) + positions (4)
5	Both	40 / 244	Same as Stages 3 and 4 respectively

In Stage 1, the thief learns basic navigation to the safe zone on the city grid, training for 100,000 episodes with only wall and distance information in its 10-dimensional observation. Stage 2 introduces 6 lethal traps and 5 traffic cars, expanding the observation to 28 dimensions. This stage is trained in two phases: Phase A uses static traps for 100,000 episodes to teach the agent to detect and route around hazards, and then Phase B switches to dynamic traps that teleport every 50 steps for another 100,000 episodes, which prevents the agent from simply memorizing fixed trap layouts. Stage 3 adds 3 randomly-walking police cars and 6 silent CCTV cameras, growing the observation to 40 dimensions and training for 200,000 episodes. The police in this stage do not learn; they perform random walks on the road cells, and the thief must learn to avoid them using the police overlay and distance features.

In Stage 4, the thief policy is frozen and executed internally using greedy argmax action selection. Two independent police agents then train their own separate PPO networks for 500,000 episodes, each receiving a 244-dimensional observation that includes the full grid map, live CCTV sighting data with staleness tracking, teammate relative position for coordination, and a 3×3 local thief detection window. Finally, Stage 5 unfreezes all three agents for simultaneous adversarial co-training over 400,000 episodes. The thief loads its Stage 3 checkpoint and the police load their Stage 4 checkpoints, and both sides adapt to each other's evolving strategies.

2.6 Surgical Weight Transplant

A key technical element of the curriculum is the weight transplant procedure that enables smooth transitions when the observation space expands between stages. Since all agents use the same network architecture, only the first linear layer changes shape when the input dimension grows (for example, from $(128, 10)$ to $(128, 28)$ between Stages 1 and 2). All other layers, including the second hidden layer, the policy head, and the value head, remain the same size and can be copied directly.

The transplant procedure works as follows. First, the new first-layer weight matrix $W^{\text{new}} \in \mathbb{R}^{128 \times d_{\text{new}}}$ is initialized with small random values drawn from $\mathcal{N}(0,\ 0.01)$. Next, the trained weights from the old first layer are copied into the corresponding column positions in the new matrix. All remaining layers are copied directly from the old checkpoint. Finally, the optimizer state is reset to fresh Adam moments, since the loss landscape changes with the addition of new input channels.

The Stage 1 to Stage 2 transplant requires careful column remapping. In Stage 1, the observation is organized as [wall patch (columns 0 to 8), distance (column 9)]. In Stage 2, the observation becomes [wall patch (0 to 8), trap overlay (9 to 17), traffic overlay (18 to 26), distance (27)]. The distance feature has moved from column 9 to column 27. A naive copy of the first 10 columns would incorrectly place the trained distance weight at the first trap channel position, so the transplant explicitly maps column 9 to column 27. The Stage 2 to Stage 3 transplant is simpler: all 28 existing columns map directly to positions 0 through 27, and the new police channels (positions 28 to 39) are initialized near zero.

2.7 Reward Structure

The reward function for the thief agent is designed to provide both sparse terminal signals for key outcomes and dense shaping signals to guide learning. Specifically, the thief receives:

$$r_t^{\text{thief}} = \begin{cases} +100 & \text{if the thief reaches the safe zone} \\ -100 & \text{if caught by police (Stages 3 through 5)} \\ -50 & \text{if the thief steps on a trap (Stages 2 through 5)} \\ -3 & \text{if hit by a traffic car (plus a 2-step stun)} \\ -1.0 & \text{if the agent bumps into a wall} \\ -0.3 & \text{if the agent voluntarily stays in place} \\ -0.1 + (d_{\text{old}} - d_{\text{new}}) & \text{otherwise (step penalty plus distance shaping)} \end{cases} \tag{6}$$

Here d_{old} and d_{new} denote the Manhattan distance from the thief to the goal before and after the current step. This structure follows the process-plus-terminal reward design used by Xu & Dang (2025), where small per-step penalties encourage efficiency and large terminal rewards mark game outcomes.

The police reward function (used in Stages 4 and 5) uses differentiated rewards for individual and team catches:

$$r_t^{\text{police}_i} = \begin{cases} +100 & \text{if agent } i \text{ directly catches the thief} \\ +50 & \text{if agent } i\text{'s teammate catches the thief} \\ -50 & \text{if the thief escapes by reaching the goal} \\ -30 & \text{if the episode times out at } T_{\max} \text{ steps} \\ -0.1 & \text{per step (encourages faster resolution)} \end{cases} \tag{7}$$

The choice to give a higher reward for a direct catch (+100) compared to a teammate's catch (+50) was motivated by the findings of Xu & Dang (2025), who demonstrate that fully shared cooperative rewards lead to "lazy pursuer" behavior where one agent reduces its effort. By differentiating the rewards, both police agents are incentivized to actively pursue while still benefiting from team success.

2.8 Hyperparameters

Table 2 summarizes the key PPO hyperparameters used across all training stages. Several parameters vary between stages based on the training context. The value clip range c is reduced from 10.0 to 3.0 in Stages 3 and 5 to prevent the gradient explosion issue described in Section 2.3. The entropy coefficient c_2 starts higher (0.05) in the adversarial stages to encourage more exploration in the face of a non-stationary opponent. The Stage 5 thief uses a lower initial learning rate of 5×10^{-5} because it is fine-tuning a pre-trained policy rather than learning from scratch.

Table 2: Key PPO hyperparameters across training stages.

	S1	**S2**	**S3**	**S4 (Pol.)**	**S5 (Thf.)**	**S5 (Pol.)**
Value clip c	10.0	10.0	3.0	10.0	3.0	3.0
Entropy c_2 (start/end)	.02/.001	.02/.001	.05/.02	.05/.005	.05/.02	.05/.02
Learning rate (start/end)	3e-4/3e-5	3e-4/3e-5	3e-4/3e-5	3e-4/3e-5	5e-5/5e-6	1e-4/1e-5
Training episodes	100K	200K	200K	500K	400K	400K

Common across all stages: $\epsilon = 0.2$, $\gamma = 0.99$, $\lambda = 0.95$, $c_1 = 0.5$, gradient clip = 0.5, 4 epochs, batch size 64, hidden dim 128.

2.9 Evaluation Methodology

To systematically measure the quality of learned policies, each stage is evaluated using periodic greedy rollouts during training and a final evaluation at convergence. During greedy evaluation, the agent selects the action with the highest probability (argmax over the policy output) at every step, rather than sampling from the distribution. This removes exploration noise and provides a clean measure of what the policy has actually learned.

Evaluation runs consist of 5 to 10 episodes conducted at regular intervals, with the frequency varying by stage: every 500 episodes in Stage 1, every 5,000 in Stage 3, and every 10,000 in Stage 4. All runs use a fixed random seed of 42 for the grid layout, ensuring that the map structure is consistent across experiments. However, agent spawn positions, trap placements, traffic lane assignments, and police starting positions are all randomized at the start of each episode to test generalization.

The primary evaluation metrics are formally defined as follows. Let N denote the total number of evaluation episodes, N_{esc} the number of episodes where the thief reaches the safe zone, N_{catch} the number where police intercept the thief, and N_{trap} the number where the thief dies to a trap. The three key outcome rates are:

$$\text{Escape Rate} = \frac{N_{\text{esc}}}{N}, \quad \text{Catch Rate} = \frac{N_{\text{catch}}}{N}, \quad \text{Trap Death Rate} = \frac{N_{\text{trap}}}{N} \tag{8}$$

These three rates, together with the timeout rate for episodes that reach $T_{\max}$ without any terminal event, account for all possible episode outcomes and must sum to 1. Additional metrics tracked include average episodic reward, average episode length, and policy entropy $H[\pi_\theta]$ which indicates the level of exploration versus exploitation in the current policy.

3 Results

3.1 Curriculum: Stages 1 through 4

Figure 2 presents the training curves for Stages 1 through 4. In Stage 1, the thief converges to approximately 98% success rate within the first 10,000 episodes, demonstrating that the distance shaping reward provides an effective learning gradient for basic navigation. In Stage 2, Phase A (static traps) reaches approximately

95% success with trap death rates dropping below 5%. When Phase B introduces dynamic traps (shown with dashed lines), performance initially dips but stabilizes around 80 to 93% success, confirming that the agent has learned to read the trap overlay in real time rather than memorizing fixed layouts.

Stage 3 shows the thief learning to evade randomly-walking police, converging to approximately 63% escape rate with a 34% police catch rate. The gap between these and 100% is accounted for by trap deaths and timeouts. Stage 4 demonstrates the police agents slowly improving against the frozen thief, with the catch rate climbing to approximately 22% while the thief maintains around 70% escape rate. The relatively slow police learning reflects the difficulty of the task: police must search a grid with over 100 road cells using only occasional CCTV sightings and 3×3 local detection.

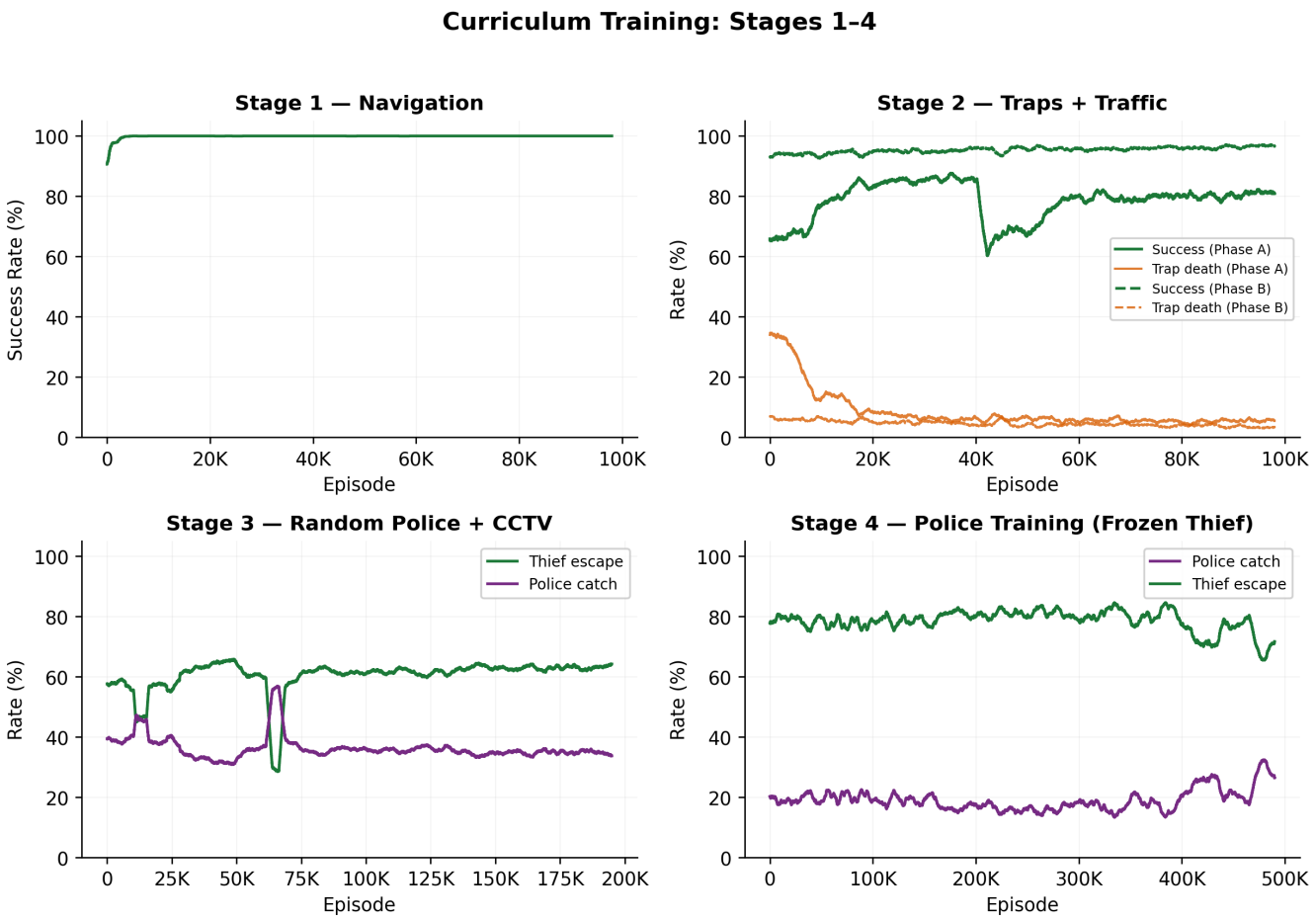

Figure 2: Training curves for Stages 1 through 4. Each subplot shows the rolling-average escape rate (green) and, where applicable, police catch rate (purple) over training episodes.

3.2 Stage 5: Adversarial Co-Training

The central result of this project is the adversarial dynamics observed during Stage 5, shown in Figure 3. Over 400,000 episodes of simultaneous training, the thief escape rate gradually declines from approximately 74% to 66%, while the police catch rate rises from approximately 23% to 32%. The trap death rate remains consistently below 4% throughout training, which confirms that the hazard avoidance behaviors learned

in Stages 1 and 2 are preserved through the weight transplant and not degraded by the adversarial training process.

This gradual and sustained shift in the outcome distribution demonstrates genuine competitive co-adaptation. Rather than one side dominating, both the thief and police continuously improve their strategies in response to the other.

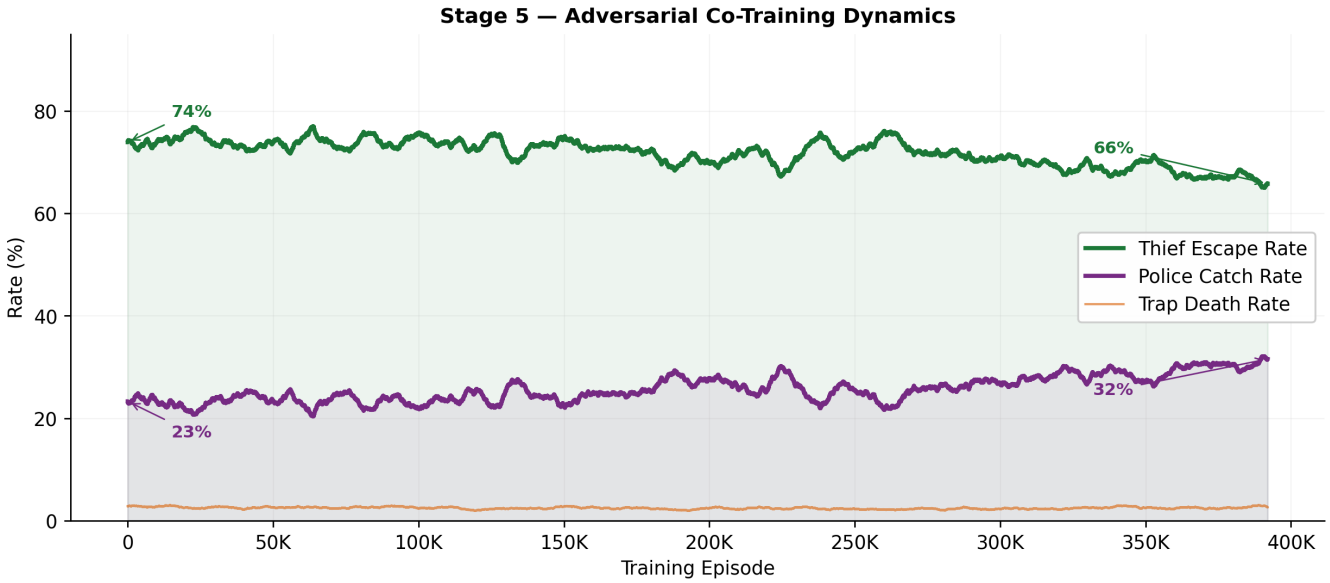

Figure 3: Stage 5 adversarial co-training dynamics over 400K episodes. The thief escape rate (green) gradually decreases while the police catch rate (purple) increases. Start and end percentages are annotated.

Figure 4 provides additional detail on Stage 5 training. The outcome breakdown (left) confirms that the three-way split between escapes, catches, and trap deaths remains relatively stable, with the main shift being a transfer from escapes to catches. The reward curves (center) show the thief's average reward declining as police improve, while the police average reward trends upward. The policy entropy plot (right) shows that all agents' entropies decrease over training, indicating that their strategies are becoming more deterministic and refined.

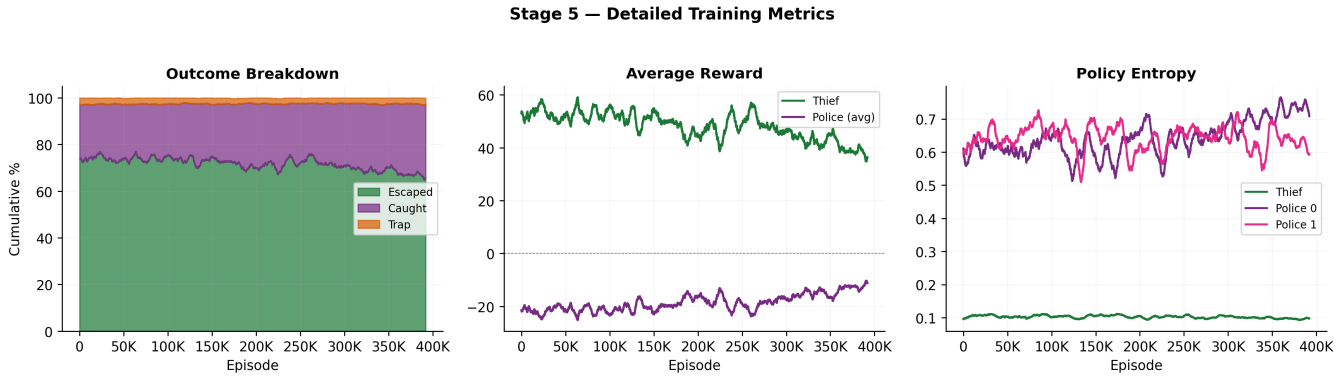

Figure 4: Stage 5 detailed training metrics: stacked outcome breakdown (left), average reward trajectories for thief and police (center), and policy entropy for all three agents (right).

3.3 Summary of Results

Figure 5 and Table 3 consolidate the final performance across all five stages.

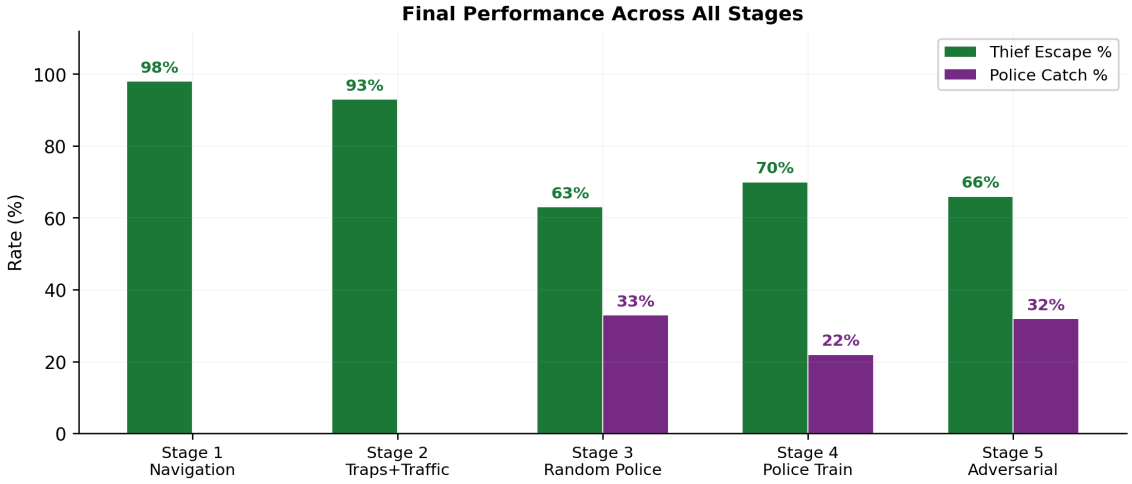

Figure 5: Final performance across all stages. Green bars represent thief escape rate, purple bars represent police catch rate.

Table 3: Summary of final results across all training stages.

Stage	Task	Metric	Value	Episodes
1	Navigation	Thief success rate	~98%	100K
2	Traps + traffic	Thief success rate	~93%	200K
3	Random police	Escape / Catch	63% / 34%	200K
4	Police training	Escape / Catch	70% / 22%	500K
5	Adversarial	Escape / Catch	66% / 32%	400K

4 Conclusions

4.1 Strengths

The curriculum-based training approach proved to be effective for this project. Each stage builds on stable behaviors acquired in earlier stages, and the surgical weight transplant technique enables smooth transitions across observation-space expansions without losing previously learned skills. The fact that trap death rates remain below 4% even in Stage 5 demonstrates that the transplanted weights successfully preserve earlier knowledge.

The Stage 5 adversarial dynamics represent the most significant outcome of this work. The police catch rate improves from 23% to 32% while the thief escape rate drops from 74% to 66% over 400,000 episodes, showing that both sides are genuinely adapting to each other. The differentiated police reward

structure (+100 for a direct catch, +50 for a teammate's catch) appears to have successfully incentivized active pursuit by both agents. This design decision was directly informed by the lazy pursuer finding of Xu & Dang (2025), who showed that fully shared cooperative rewards can cause one agent to free-ride on the other's effort.

4.2 Limitations

The police catch rate of 32% at the end of Stage 5 is notably lower than the 99.9% pursuit success reported by Xu & Dang (2025) and the 85.2% achieved by Lehis et al. (2025). There are several factors that explain this difference. Our information asymmetry gives the thief a structural advantage, since it has immediate local knowledge of nearby threats while police must rely on stale CCTV data. We use only 2 police agents compared to 10 in the work of Lehis et al. (2025). Additionally, our environment contains traps and traffic that do not exist in either reference work, creating additional sources of episode termination that are unrelated to the pursuit-evasion contest itself.

Another limitation is the single-episode rollout strategy used for PPO updates. While this approach works well for episodes of up to 200 steps, it would not scale efficiently to longer game durations. A fixed-length rollout buffer of 2,048 or 4,096 steps, as is common in large-scale PPO implementations, would be more appropriate for extended scenarios.

Software and Tools

All code was written in Python 3.10 using PyTorch (Paszke et al., 2019) for neural network implementation and training, NumPy for numerical computation, and Matplotlib for plotting and visualization. The grid-world environments and the PPO training algorithm were implemented from scratch without using external reinforcement learning frameworks such as Gymnasium. Claude (Anthropic) was used to assist with code structuring and commenting. All source code, training scripts, and plotting utilities are included in the submitted repository.

References

Lehis, S., Siam, A., Moumen, H., Chergui, W., Souidi, M. E. H., & Bekhouche, A. (2025). Multi-Head DDPG for Pursuit-Evasion with Interpretable Behavioral Decomposition. *Ingénierie des Systèmes d'Information*, 30(12), 3117–3130.

Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 32, 8026–8037.

Xu, S. & Dang, Z. (2025). Emergent behaviors in multiagent pursuit evasion games within a bounded 2D grid world. *Scientific Reports*, 15, 29376.